The Ledger and the Store: Tape, Dolt, the Object Graph, and Why Reads Got Slow

Learning series, document 1 of 6. 14 October 2026. Repository: `origin/main` at 980ffaf4e. Written for Choir's owner, to teach the storage layer from first principles and then show exactly how Choir builds it, where it hurts, and what the choices are. Mutation class of this document: **green** (docs only). Every option it discusses names its own class; most store changes are **orange** (queries, guest schema) and anything touching the event chain or Store A is **red**.

Evidence labels used below: **code** (read in the repository at the commit above, with a path), **doc** (an existing Choir document, linked), **upstream** (Dolt or general database documentation, explained from first principles), and **inference** (reasoning that has not been tested; treat it as a hypothesis). This series used no live host access: nothing on Node B or staging was read for it. Measured numbers come from the reports and problem records they cite.

How to read this

Section 1 is a primer on logs, hash chains, compare-and-swap and event sourcing: the ideas the tape is built from. Section 2 is the tape itself, traced through one append. Section 3 is the commitment ledger that rides on the tape. Section 4 teaches Dolt: prolly trees, commits, `AS OF`, and how Dolt differs from MySQL's InnoDB. Section 5 is the object graph: kinds, edges, the registry and the write gate. Section 6 sorts out the three stores (Store A, Store B and the guest's embedded store) and why they are separate. Section 7 is today's CPU finding: why reads got slow. Section 8 teaches indexes and query plans, using Choir's own incidents. Section 9 is the options table. The document ends with a glossary, five self-check questions, the three files to read next, and sources.

If you only have twenty minutes, read "The short answer", section 2.3 (one append, step by step), and section 7.

The short answer

1. **The tape is the truth; every table is a view.** A computer's canonical history is an append-only, hash-chained sequence of events (`internal/computerevent`). The guest's embedded Dolt database is a *materialization* of that tape, rebuilt by replay, never an alternate authority (doc: computer ontology, "Dolt Store Taxonomy"). This is event sourcing.
2. **One writer per computer, arbitrated by a compare-and-swap.** Only the guest's `ComputerEventAppender` appends; corpusd accepts an append only if the computer's head in Store A still equals the head the event names (`internal/platform/computer_events.go`, `CompareAndSwap`). That single conditional `UPDATE` is what makes the chain linear.
3. **Every object write in the guest is an event.** When the tape is bound, `og_objects` writes do not go to SQL directly; an interceptor wraps them as a `projection_batch_recorded` event, appends it, and only then applies it to Dolt (`internal/store/projection_tape.go`). So the cost of a write is the cost of an append: about five host round trips and two serializable transactions (doc: CPU efficiency §1c).
4. **The commitment ledger is records on that tape.** Precommits, reports, resolves, disagreements and directives are typed `CommitmentRecord`s (`internal/types/commitment.go`) stored as `choir.commitment_record` objects. Since the inbox work, they are also the message substrate between desks, which is why reading them became the hottest path in the system.
5. **Dolt is "Git for tables" built on prolly trees.** Content-addressed chunks give cheap diffs, branches and `AS OF` time travel. The price is write amplification and history that garbage collection cannot reclaim while any commit references it. Choir learned this the hard way: 6.9 million per-mutation commits grew the platform store to 218 GB (doc: 218 GB evidence). Choir now commits on a debounce and barely uses Dolt's history features at all: no production query uses `AS OF` (code, by search).

6. **There are three Dolt stores, not two.** Store A is the host's *platform control* store (event heads, idempotency, lifecycle, routes; port 13306). Store B is the host's *corpus* store: the Autopaper object graph of captures and publications (port 13307). Each guest has its own *embedded* store. Note the naming: Store A is not "the guest".
7. **The object graph is a typed property graph in two tables.** Objects and edges have kinds; since H-OG1 (7f287a135) every durable write goes through one gate, `objectgraph.CheckWrite`, that refuses kinds the registry does not know.
8. **Reads got slow because the graph is used as a JSON document store.** 73% of the paper computer's main-process CPU was the embedded Dolt engine, and 52% was one access path that filters every commitment record by a JSON field no index can see (doc: guest CPU is unindexed store scans). The cost grows with records ever written, not with work done.
9. **The fix is an access path, not a patch.** An indexed replacement half exists (`metadataIndexColumns`, generated columns) but is wired into only the unpaged read. Wire it in, add the missing fields, then move hot reads onto typed projection tables, and guard every shipped query with a plan-shape test, because Dolt's planner can turn 1.5 ms into 12 s when one index is added (doc: retrieval research §4 G).

1. Primer: logs, chains, compare-and-swap and event sourcing

1.1 The append-only log

A **log** is the simplest durable data structure: a sequence of records, each with a position, where the only write is "add one at the end". Logs matter because they turn the hardest problem in distributed systems (several parties agreeing on the current state) into an easier one (several parties agreeing on the *order* of changes). If everyone applies the same changes in the same order to the same starting state, using a deterministic function, everyone reaches the same state. This is the **state machine replication** idea behind database write-ahead logs, Kafka, Raft and Git.

Three properties make a log useful as an authority:

property	what it means	what breaks without it
append-only	records are never edited or deleted	history can be rewritten; audits and replays disagree
totally ordered	every record has one position	two replicas apply concurrent changes in different orders and diverge
deterministic reduction	`state = fold(reducer, log)` gives the same answer every time	replay produces a different state than the one that was served

The third is subtle. It means the reducer must not call a model, read the clock, or read the network. Choir's ontology says this directly: reconstruction "deterministically rebuilds embedded state, and never reruns a model, tool, or network observation" (doc: computer ontology). Anything non-deterministic (a model reply, a fetched page) is recorded *as data* in an event, so replay reads it instead of recomputing it.

1.2 Hash chains and heads

A **hash chain** links each record to the one before it by including the previous record's cryptographic hash. Record *n* contains `hash(record n−1)`; the hash of record *n* therefore commits to the entire history before it. Change one byte anywhere in the past and every later hash changes. The hash of the newest record is the **head**: a 32-byte name for the whole history. Git commits, Bitcoin blocks and certificate transparency logs all work this way.

Hashes only work if both sides compute them over the same bytes. JSON is not canonical by default: key order, whitespace and number formats vary. So systems that hash JSON use a **canonical encoding**. Choir uses RFC 8785 (JSON Canonicalization Scheme) and rejects floating-point numbers so platform-dependent float rendering cannot enter a signed preimage (code: `internal/computerevent/canonical.go`, `CanonicalJSON`).

1.3 Compare-and-swap and optimistic concurrency

If two writers append at the same time, both may compute "next position = 42" and both may succeed. The classical fix is **compare-and-swap (CAS)**: "set the head to X only if it is still Y". In SQL that is a conditional update:

```
UPDATE heads SET head = :new, seq = :seq_plus_1
WHERE  id = :id AND seq = :seq AND head = :old;   -- 1 row = won, 0 rows = lost
```

The loser sees zero rows affected, re-reads the head, and retries or refuses. This is **optimistic concurrency**: assume no conflict, detect it at commit. The alternative, pessimistic locking, holds a lock across the whole read-modify-write. Optimistic control is cheaper when conflicts are rare and the writer can cheaply redo its work.

A related tool is the **idempotency key**: a client-chosen identifier for an operation, stored with its result, so that a retry of the same operation returns the original result instead of applying twice. Network calls fail ambiguously (did the server commit before the connection dropped?), so any write that crosses a network needs one.

1.4 Event sourcing and materialized views

Event sourcing stores the log of changes as the source of truth and derives current state from it. The derived tables are **materialized views** or **projections**. They are caches: if you lose them you rebuild them by replaying the log. Two consequences follow, and Choir lives with both:

- **Reads are only as good as the projection's shape.** The log is ordered by time; questions are asked by subject ("what is open in this conversation?"). If no projection is shaped for the question, every read scans. Section 7 is this failure.
- **Replay cost grows with history.** A long log needs **snapshots**: a saved projection at some position, so recovery replays only the tail. Choir calls these projection bases (section 2.6).

1.5 The picture

```
writer (one per computer)
     |  event n: { prev_head = H(n-1), seq = n, kind, payload_commitment, ... }
     v
+------------------------- the log (tape) -----------------------------+
| e1 <- e2 <- e3 <- ... <- e(n-1) <- e(n)        head = sha256(e(n))    |
+-----------------------------------------------------------------------+
     |  deterministic reducer: state = fold(apply, events)
     v
+---------------- projections (materialized views) --------------------+
|  og_objects, og_edges, run memory, desktop state, Texture revisions   |
|  -- a cache of the log; rebuildable; never an alternate authority --  |
+-----------------------------------------------------------------------+
     |  queries
     v
   desks, API, Texture surface
```

2. The tape in Choir

2.1 The event envelope

The tape's record type is `computerevent.Event` (`internal/computerevent/event.go`). It is a "complete V1 semantic event envelope": 29 serialized fields, of which the chain-critical ones are:

field	role
`ComputerID`	which computer's chain this belongs to
`Sequence`	position; must be the current head's sequence plus one
`PreviousHead`	the hash of the prior event: the chain link

field	role
`EventKind`	one of 26 kinds (`trajectory_started`, `effect_accepted`, `projection_batch_recorded`, `key_revoked`, ...) checked against `validEventKinds`
`IdempotencyKey`	the retry identity
`RequestCommitment`	a hash binding the event to its inputs and payload pin receipts
`PayloadCommitment`, `PrivacyClass`	the payload is stored separately, by hash; the event holds the commitment
`ExpectedDesiredEventHead`, `ExpectedEffectiveEventHead`, `ExpectedPendingTransitionRef`, `Expected...StateCommitment`	preconditions on the computer's *state*, not just its position

The event body contains no digest of itself. `Event.Digest()` computes `sha256(CanonicalBytes())`, and that digest becomes the next head. Event IDs are UUIDv7 (`NewEventID`), which sort by creation time.

The payload is not in the event. Large or private bodies (a model reply, a projection batch) are **pinned** into content-addressed storage on the host first; the event carries their hash. Privacy-class payloads are encrypted before pinning (`AppendNewPrivatePayload`, `internal/computerevent/privacy.go`). This keeps the chain small and lets the host store bodies without being able to read private ones.

2.2 The head is more than a position

A naive log head is "sequence and hash". Choir's head carries the computer's state machine too (`internal/computerevent/reducer.go`, `Head`):

```
Head {
  Sequence, CanonicalEventHead          -- where the chain is
  DesiredEventHead, DesiredStateCommitment     -- what the computer should become
  EffectiveEventHead, EffectiveStateCommitment -- what it has actually become
  PendingTransitionRef                  -- a materialization in flight, if any
  ReducerVersion, CredentialRevocationEpoch
}
```

Desired versus effective is the important split. An accepted effect (say, a self-development change) moves the *desired* head; only after the guest updater stages and health-checks the release does a `materialization_applied` event move the *effective* head (doc: computer ontology). `Reduce` enforces the transitions: an `effect_accepted` event is refused while a transition is pending (`ErrPendingTransition`), and must carry a target state commitment, a proposed effect, a decision and at least one verifier reference (`reducer.go`, `Reduce`). This is how "the computer says it changed" is kept separate from "the computer did change". Document 4 covers the lifecycle side.

2.3 One append, step by step

`ComputerEventAppender.appendLocked` (`internal/computerevent/appender.go`) is the heart of the system. The type's comment states the authority rule: it is "the sole semantic event sequencer in the trusted guest core ... agents, capsules, reducers, vmctl, and route projections never receive this object or its append capability." Here is what one append does, in order (code):

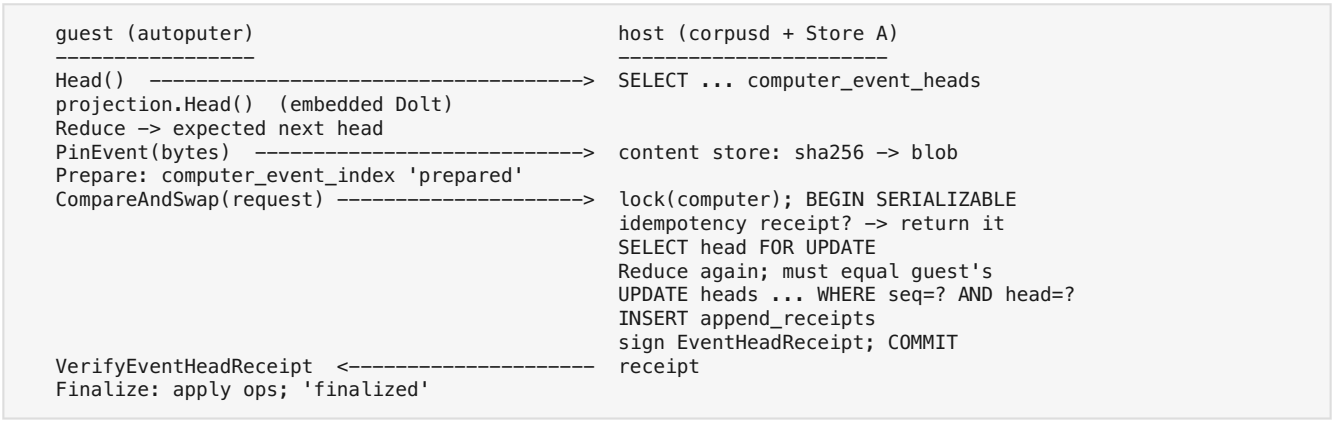
1. **Vocabulary fence.** `vocabmigrate.VerifyServingVocabulary` refuses an event whose actor fields are not in the live V2 vocabulary.
2. **Commitments.** Recompute the pin-intent commitment and request commitment from the event and its inputs; refuse on mismatch.
3. **Read both heads.** Ask the host for the platform head (`a.cas.Head`) and the guest's projection for its head (`a.projection.Head`). If they differ, refuse with `ErrNeedsProjectionRepair`. The guest will not write on top of a projection it knows is stale.
4. **Reduce locally.** `Reduce(platformHead, event, input)` computes the next head the guest expects.
5. **Pin the event.** Canonical bytes go to the host's content store (`PinEvent`); the returned artifact digest must equal the event digest.

6. **Prepare the projection.** `a.projection.Prepare` writes the event into the guest's `computer_event_index` with `status='prepared'` (`internal/store/computer_events.go`). This is the first half of a two-phase commit.
7. **CAS on the host.** `a.cas.CompareAndSwap(request)`. On failure, discard the prepared row.
8. **Verify the receipt.** The host returns a signed `EventHeadReceipt`; the guest verifies the signature and its binding to the request (`VerifyEventHeadReceipt`).
9. **Finalize.** Apply the projection and flip the row to `status='finalized'` (`finalizeProjection`). If this fails, the error is `ErrNeedsProjectionRepair`: the host has accepted the event, so the guest must repair forward, never pretend it did not happen.
10. **Notify.** A post-commit observer runs detached (`go (*observer)(kind)`); it can never block or alter the commit.

`RecoverPrepared` handles a crash between steps 6 and 9, and it is a small masterclass in idempotency. At boot, for each `prepared` row it reads the host head and asks one question: did the CAS happen?

- **Host head equals this event.** The CAS committed but the guest died before finalizing. The guest calls `CompareAndSwap` *again* with the same request; the host's idempotency lookup returns the original signed receipt instead of appending twice; the guest verifies it and finalizes.
- **Host head is this event's predecessor.** The CAS never committed. Discard the prepared row; the work is simply not on the tape.
- **Anything else.** Refuse with `ErrNeedsProjectionRepair`.

During replay, prepared rows are discarded outright, because "the replay path NEVER performs a semantic CAS (it must not become a second writer)" (code comment, `RecoverPrepared`).



2.4 The CAS on the host

`ComputerEventCAS.CompareAndSwap` (`internal/platform/computer_events.go`) is worth reading line by line, because it stacks four independent defences:

1. **Recompute everything the guest claims.** The pin-intent and request commitments are recomputed; the event's digest is recomputed; the pins are validated (`ValidateEventPins`). The host trusts no hash the guest sends.

2. **A per-computer in-process lock** (`c.locks[computerEventLockIndex(...)]`, a striped array of mutexes) serializes appends for one computer inside corpusd.

3. **Idempotency first.** `existingComputerEventReceipt` looks up `(computer_id, idempotency_key)` in `computer_event_append_receipts`. A retry with the same request commitment gets the original receipt back; a retry with a *different* commitment under the same key is a conflict. It checks once before the transaction and again inside it.

4. **A serializable transaction with a conditional update.** It reads the head `FOR UPDATE`, runs the same `Reduce` the guest ran, and refuses if the result differs from the guest's `Next` ("guest projection mismatch"). Then:

   ```
   UPDATE computer_event_heads SET sequence=?, canonical_event_head=?, ...
   WHERE computer_id=? AND sequence=? AND canonical_event_head=?
   ```

   Anything other than one affected row is `ErrComputerEventCASConflict`.

The schema backs this up with constraints: `computer_event_heads` has one row per computer (primary key `computer_id`) and `CHECK` constraints on hash lengths; `computer_event_append_receipts` has unique keys on `(computer_id, sequence)` and `(computer_id, event_digest)`. Even if every code check above were wrong, the database would refuse a second event at the same position.

Notice what the host does *not* do: it never decides which events a computer should have. The ontology calls this "mechanical" CAS: corpusd "does not select semantic events" (doc: computer ontology). Authority to *say* something lives in the guest; authority to *order* it lives on the host. That split is the single-state-authority principle document 6 discusses.

2.5 Every object write is an event

In production, the guest store binds a **projection tape** (`internal/store/projection_tape.go`, `BindProjectionTape`). It installs `interceptOG` as the object graph's `MutationInterceptor`. From then on, `DoltStore.PutObject`, `PutBatch` and `PutEdge` do not write SQL. They marshal each object and edge into a `ProjectionOp`, wrap the ops in a `ProjectionBatch`, and call `AppendNewPayload` with a `projection_batch_recorded` event (`appendOps`). The batch is the payload; the event pins it, CASes the head, and finalization applies the ops to `og_objects`/`og_edges` through the projector (`internal/store/project.go`, `projectObject`, `projectObjectEdge`).

This is the design's strongest property and its biggest cost:

- **Strength.** There is exactly one way for durable guest state to change, and it leaves a hash-chained, host-ordered, signed record. A lost or corrupted embedded store is rebuilt from the tape. Restore to an earlier head is replay to that head.
- **Cost.** A small object write costs an append. The CPU report counted, per write: five HTTP round trips to corpusd (two head reads, `PinNonPrivatePayload`, `PinEvent`, `CompareAndSwap`), about six to ten canonical-JSON passes, one ed25519 verification, a dry-run serializable transaction that applies the projection and rolls back, an insert, and a second serializable transaction that applies it again (doc: CPU efficiency §1c). The measured share was 8 to 10% of the main process, plus about 0.18 cores of host work. That is significant but not the main cost; section 7 is.

The interceptor runs *outside* the engine mutex: "intercept is the projection append path (network CAS + event write) and must not hold the engine mutex across a host round-trip" (`dolt_store.go`, `PutObject`). Holding a local lock across a network call is a classic way to turn a slow peer into a frozen process.

2.6 Replay, projection bases and compaction

Replaying a long tape is slow, so Choir snapshots. `internal/projectionbase` builds a **projection base**: an offline rebuild, outside the guest VM, that "publishes a content-addressed ProjectionBase blob. Routine runs seed from the latest compatible verified published base and replay only its tail" (`rebuilder.go`). Installing a base verifies it before use: descriptor bindings, a watermark not above the head, a blob digest match, the unpacked store's head equal to the watermark, and tail ancestry from the immutable tape (`install.go`, `InstallVerifiedBase`). A content **witness** (`witness.go`) is a digest of the schema and per-table content, ignoring operational timestamps, so a seeded base and a genesis replay of the same chain can be compared.

`CompactWorkspace` (`compact.go`) squashes the projection store's Dolt history to one commit and runs a full GC, refusing unless the witness is identical before and after. Its comment states the principle: "The projection store is a cache of the tape (O21) and nothing reads its Dolt history." The owner store measured 8.9 GiB of which history and old chunk generations were 81%; `dolt gc --full` after squashing took it to 1.7 GiB (doc: guest store history bloat).

2.7 Where the tape is weak

weakness	evidence	why it matters
Per-write overhead. Five round trips, two serializable transactions, many JSON canonicalizations per small object write.	doc: CPU efficiency §1c, levers C6/C7	Write latency and host CPU scale with object writes, which agents produce in bulk.
Replay page scanned the whole chain. `WHERE computer_id=? AND sequence>? ORDER BY sequence LIMIT ?` took 3.6 s even when it returned zero rows, because Dolt chose the primary key `(computer_id, idempotency_key)` and ignored the unique `(computer_id, sequence)` index. It caused a deploy rollback.	doc: replay page scans; fixed in code as point lookups `sequence IN (...)` (`internal/platform/event_replay.go`)	A query planner choice turned recovery into a 34-minute operation. Section 8 explains why.
Per-run sequence by scan. `Store.AppendEvent` loads up to 10,000 earlier events of the run with bodies and decodes each to find `max(seq)` (`internal/store/store.go`, `AppendEvent`). `StreamSeq` already moved to an in-memory high-water mark (`maxStreamSeq`); `Seq` did not.	doc: CPU report §1c, measured 4.5 ms per append at 1-50 prior events rising to 25.6 ms at 400-800	O(n) per append is O(n^2) per run.
Check-then-act around the interceptor. In `putBatch`, preconditions (`ObjectCondition`) are evaluated under the engine lock, the lock is released, and then the interceptor appends. Nothing at this layer re-checks the conditions when the batch is applied.	code: `internal/objectgraph/dolt_store.go`, `putBatch`	Atomicity of a conditional write in production depends on callers holding their own lock across the call. `CommitInboxRecords` does (it holds `trajectoryMu` across planning and `PutBatchConditional`, `internal/store/inbox.go`). **Inference:** a caller that does not has a time-of-check/time-of-use window. Worth a test.
Unbounded receipts. `computer_event_append_receipts` grows one row per event with no declared retention.	doc: Store A dead data and history	Idempotency needs some of it; nobody has said how long.

3. Commitment records: the ledger on the tape

3.1 Primer: why freeze a prediction

A capable model can explain almost any outcome after the fact. If you ask it, after a failure, what it expected, it will mix what it expected with what it now knows. The **precommitment record** fixes that by freezing the prediction before the evidence exists: hypothesis, assumptions, alternatives, probabilities. When the outcome arrives, a separate record resolves and scores it. The memo grounds this in Dawid's *prequential* principle: judge a forecaster only by forecasts made before outcomes are seen (doc: Precommitment Records — Engineering Memo).

For a storage engineer, this is a requirement on the ledger, not just a product idea. A precommitment is worthless unless you can prove it existed *before* the evidence. That needs append-only storage and a trustworthy order. The tape provides both: a record's position in a hash chain whose head the host signed is evidence of when it was written. **Inference:** a proof of pre-outcome ordering also needs the evidence's own arrival time on the same or a comparable clock; the state report notes that "pre-outcome ordering is not proven" yet (doc: state and vision §2.5).

3.2 The record type

`types.CommitmentRecord` (`internal/types/commitment.go`) is a large, additive type. The parts that matter for storage:

- **Identity:** `RecordID`, which is also the object's canonical-ID suffix; `SchemaID` pins the body schema version.

- **Taxonomy:** `Kind` is one of `precommit`, `report`, `resolve`, `disagreement`, `directive` (`CommitmentKind*` constants). The comment says it "duplicates what the typed sub-objects imply so routing and projection classify without decoding intent names out of the RecordID string." That is a denormalization for read speed, done right.
- **Typed bodies:** `Precommit`, `Resolve`, `Disagreement`, `Directive`, `Cast`, `Report`, `Support`, `Captures`, `Disposition`, `TaskOutcome`.
- **Inbox addressing:** `Inbox` (for example `texture:<doc>`, `management:<owner>`), `SenderInbox`, `InboxSeq` (the next number in that inbox, stamped by the store at commit), `ConversationID` (derived from the unordered inbox pair), `Round`, `Final`.
- **Reserved for cross-computer scoring:** `EmitterKind`, `ComputerID`, `SubjectRefs`.

Records are written as `choir.commitment_record` objects, through the object graph and therefore through the tape (writers in `internal/store/lifecycle_commit_act.go`).

3.3 Records became the message bus

Since the P1-P2 protocol work, every inter-desk message is a commitment record (Cast, Ask, Note, Report, Resolve) rather than a free message (doc: state and vision §3.3). That is a good authority decision: every message is now a durable, attributable, ordered commitment. But it changed the *access pattern*. Records are now read like a mailbox, constantly:

- `ListInboxRecords` reads an inbox's records by sequence (`internal/store/inbox.go`);
- `ConversationState` folds one conversation's records into its head;
- `OpenRounds` finds every conversation an inbox takes part in, then folds each one;
- `ListCommitmentClaimRecords` reads the owner's records to build the acting desk's claim pack (`internal/store/commitment.go`).

A ledger optimized for "append, and occasionally audit" became a queue read on every desk run and every cell. Section 7 measures what that cost. Document 2 covers the inbox and wake mechanics from the desk side.

3.4 Where the ledger is weak

- **Nothing settles or scores on the live paper yet.** The typed record, act taxonomy, append-once persistence and derived views exist; settlement, scoring and context-pack injection do not (doc: state report §2.5).
- **Reads are scans.** Inbox, conversation and claim lookups filter JSON fields of every record (section 7).
- **Schema is in Go, not in the database.** The registry records `Schema: "types.CommitmentRecord"` (`internal/objectgraph/registry.go`), but the database stores a blob and a JSON string; nothing in SQL validates a record's shape.

4. Dolt from first principles

4.1 What a normal database does: B+trees and update in place

MySQL's InnoDB stores each table as a **B+tree** keyed by the primary key (the *clustered index*): fixed-size pages (16 KB by default), sorted keys, rows in the leaves. A secondary index is another B+tree whose leaves hold the indexed columns plus the primary key, so a lookup by secondary index is two tree walks: index, then table (upstream, general). Writes update pages **in place**, protected by a redo log for crash safety and an undo log for multi-version concurrency. Old versions are kept only as long as some open transaction needs them; then they are purged. InnoDB remembers the present, not the past.

4.2 Prolly trees: a B-tree you can hash

Dolt stores tables in **prolly trees** ("probabilistic B-trees") (upstream: Dolt prolly tree architecture). The ideas:

1. **Rows sorted by key, cut into chunks by content.** Instead of fixed-size pages, a rolling hash runs over the sorted key/value stream and declares a chunk boundary wherever the hash hits a pattern. Chunks come out around a target size (a few KB). Because boundaries depend only on content, **the same rows always produce the same chunks**, regardless of the order they were inserted. This is *history independence*.
2. **Chunks are named by their hash.** Each internal node lists its children's hashes, so the root hash names the whole table, the way a Git tree hash names a directory. Storage is **content-addressed**: a chunk is stored once no matter how many versions refer to it.
3. **Structural sharing.** Changing one row rewrites its leaf chunk and the path of parents up to the root (each parent's list of child hashes changed). Every other chunk is shared between the old and new version.
4. **Cheap diff.** Comparing two versions walks both trees and skips any subtree whose hash is equal. A diff costs O(changes), not O(table).

```
version 1 root R1                    version 2 root R2 (one row changed)
     /      \                             /      \
   A1        B1                         A1        B2      <- B rewritten
  /  \      /  \                       /  \      /  \
 a    b    c    d                     a    b    c    d'   <- d rewritten
shared: A1, a, b, c          new chunks: R2, B2, d'
```

What this buys: commits, branches, diffs, merges and time travel for almost nothing at read time. What it costs:

- **Write amplification.** One small update writes a new leaf and a new chunk at every level above it. **Inference:** for a table of a few hundred thousand rows that is a handful of chunks of a few KB each per write, against InnoDB's in-place page update plus log record.
- **Garbage that cannot be collected while referenced.** The old chunks (d, B1, R1) are garbage *only if* no commit references R1. If you commit after every write, every version stays reachable forever.
- **A Go query engine.** Dolt's SQL layer is go-mysql-server, written in Go. Expressions such as `JSON_EXTRACT` are evaluated per row in Go; in the paper's profile that shows up as `encoding/json.Unmarshal` (doc: CPU report §1b). This matters in section 7.

4.3 Working set, SQL COMMIT and DOLT_COMMIT

Dolt has two different "commits", and confusing them caused Choir's largest storage incident.

operation	what it does	Choir's use
SQL `COMMIT`	ends a transaction; the change is durable in the **working set**	every write; durability "verified on Dolt 2.1.9 by the 2026-09-19 kill-9 drill" (code: `internal/doltbatch/committer.go`, package comment)
`CALL DOLT_COMMIT(...)`	records a new **Dolt commit** (root hash + parents + message) in the commit graph, like `git commit`	a snapshot for history and `AS OF`; optional for durability

Until August, `internal/platform/store.go` and the old cycle store called `DOLT_COMMIT('-Am', ...)` on essentially every mutation. The platform store accumulated 6,903,253 commits and 218 GB in `.dolt/noms/oldgen` against about 4.5 GB of live data. GC could not help: "Dolt GC — shallow *or* `--full` — preserves everything reachable from any commit on any branch" (doc: 218 GB evidence). Note also the record of how the root cause was found: eleven of twelve consensus models approved the wrong hypothesis (shallow GC), and the one dissent was right.

The fix is `internal/doltbatch`: mutations call `markDirty`, and one committer per database runs `DOLT_COMMIT` on a debounce (default 45 s, `DefaultDebounce`, overridable by `DOLT_SNAPSHOT_DEBOUNCE`), or promptly at recovery boundaries such as checkpoint publication and replay watermarks (`CommitNow`). In the guest, `defaultFinalizeCheckpointEvery = 100`: one Dolt commit per hundred finalized events, after 138k tape rows and about 6k commits had backed a 9.8 GB journal for about 0.5 GB of live data (code: `internal/store/store.go`).

4.4 AS OF, history tables, branches and merges

Because every Dolt commit names a root hash, you can query any of them (upstream: querying history):

```
SELECT * FROM og_objects AS OF 'my-tag' WHERE ...;      -- the table as it was
SELECT * FROM dolt_history_og_objects WHERE canonical_id = ?;  -- every version of a row
SELECT * FROM dolt_diff('HEAD~1', 'HEAD', 'og_objects');       -- what changed
CALL DOLT_BRANCH('experiment'); CALL DOLT_MERGE('experiment'); -- Git-style
```

Merges are three-way, cell by cell, with conflicts recorded in system tables. The retrieval research measured `AS OF` a tag as fast as a head read (8.6 ms p50 versus 22.5 ms at head in one test) and a day's `DOLT_COMMIT` at 0.29 s (doc: retrieval research §4 F).

How much of this does Choir use? Almost none. A search of non-test code under `internal/` and `cmd/` finds no `AS OF`, no `dolt_history_` or `dolt_diff` query, no `DOLT_BRANCH`, `DOLT_MERGE` or `DOLT_TAG` call (code, by search; the only matches are comments). Texture history "follows the revision parent chain", and the comment on `getHistory` says the chain, "not Dolt commit history or AS OF snapshots — is the history authority" (`internal/store/texture.go`). The ontology records that the old tag-based promotion adapter and its destructive `DOLT_RESET --hard` rollback were deleted (doc: computer ontology).

This is a deliberate and defensible position: **the tape is the history**, so Dolt's history is redundant in the guest and dangerous on the host. The open question is Store B. There, Dolt's history *could* be the natural home of "what did the Autopaper object graph say at time T", which the claims design needs for scoring. The retrieval research recommends append-only rows with indexed timestamps as the primary mechanism and `DOLT_TAG` witnesses at publication and settlement boundaries, with rebuildable projections in `dolt_ignore`d tables so they never enter history (doc: retrieval research §0 item 7, §3.1).

4.5 Embedded versus server, and the one connection

Choir runs Dolt two ways:

- **On the host, as servers.** `dolt sql-server` on 127.0.0.1:13306 (Store A) and 127.0.0.1:13307 (Store B), each its own systemd unit (code: `nix/node-b.nix`). corpusd connects with the MySQL driver.
- **In the guest, embedded.** The autoputer daemon links Dolt as a library through `github.com/dolthub/driver/v2` and opens workspaces from files (`internal/store/texture.go`, `embedded.NewConnector`). No separate process, no network hop.

Embedding is simple and fast for one process, but it shares one engine among every goroutine. The guest sets every embedded pool to one connection (`configureEmbeddedDoltDB`: `SetMaxOpenConns(1)`), and every object graph call takes a process-wide `engineMu` (`internal/objectgraph/engine_mutex.go`, which also records wait and hold time per caller). The reason was a real data race: Dolt's `unescapeHTMLCodepoints` mutated a JSON buffer that aliased a shared, cached prolly node (doc: embedded Dolt single connection). Upstream fixed it on 16 July in 6bea93f2f665; Choir upgraded past it in 5b0772561, and `go.mod` now pins `github.com/dolthub/dolt/go v0.40.5-0.20260716231223-6bea93f2f665` (code). The problem record's plan was: upgrade, then "raise the connection count, and remove the engine mutex in measured steps". **The first step landed; the second has not** (code: `SetMaxOpenConns(1)` and `engineMu` are both still present).

The consequence is that every read and write in the guest's object graph queues on one lock. An earlier measurement found `engineMu` held about 97% of wall time over five minutes under desk drain (doc: retrieval research §1.4, citing texture latency layered evidence). A slow query is therefore not just slow; it delays everything behind it, including health and the Texture editor.

4.6 Options and trade-offs for the engine

option	gains	costs	class
Keep one connection; make queries cheap (section 9)	no concurrency risk; fixes the root cause	lock contention remains for long reads	orange
Raise connections now that the race fix is in	parallel reads	needs the race test the problem record specifies; Dolt's transaction semantics under concurrent embedded sessions are less exercised than the server's (**inference**)	orange
Move hot projections out of Dolt (SQLite file, rebuildable)	a mature engine for queue-shaped reads	two engines in the guest; must stay derived from the tape	orange
Run the guest store as a `dolt sql-server` process	crash isolation; server-tested concurrency	memory per guest; a local network hop	orange/red

5. The object graph

5.1 Primer: a property graph on two tables

A **property graph** is nodes and edges, each with a type (a *kind*) and properties. You can store one in a relational database with two tables: one row per node, one row per edge. The design question is where the properties go:

approach	example	reads	schema change
typed tables per kind	`captures(url, host, fetched_at, ...)`	fast, indexable	migration per change
generic rows + JSON properties	`objects(kind, id, metadata JSON, body BLOB)`	fast only by id/kind; JSON filters scan	none
entity-attribute-value	`(id, attribute, value)` triples	joins everywhere	none

Generic rows plus JSON is what document stores do, and it is the fastest way to build. Its cost arrives later: every new question about a property inside the JSON is a scan, unless you add an index on an *expression* over the JSON. Choir chose generic rows plus JSON. Section 7 is the bill.

5.2 Choir's schema

`objectgraph.SchemaDDL` (`internal/objectgraph/dolt_store.go`) defines, identically in the guest and on corpusd ("so the same queries work in both environments"):

```
og_objects(canonical_id PK, object_kind, owner_id, computer_id,
           version_id, content_hash, body LONGBLOB, metadata LONGTEXT,
           created_at, updated_at, tombstone, superseded_by)
og_edges  (edge_id PK, from_id, to_id, kind, metadata LONGTEXT,
           created_at, tombstone)
```

Store B adds `body_ref` and `body_size`: bodies over 1 KiB live in a filesystem content store and the row keeps a reference (`internal/platform/store.go`, `og_objects`; `SetArtifactsRoot`). Capture bodies are deliberately kept out of Dolt rows "to avoid repeating the 104 GB store of the old ingestion system" (doc: state report §3.4).

Canonical IDs have the form `obj:<kind>:<owner>:<suffix>` (`internal/objectgraph/object.go`, `BuildCanonicalID`). Because the kind is a prefix of the primary key, rows of one kind are contiguous in primary-key order. That makes "all objects of kind K" a primary-key range scan.

Identity modes (`registry.go`): `content_hash` (the ID is derived from the content, so the same content is the same object; captures, revisions, events) and `external_key` (the ID is a stable name; runs, documents, work items). `Versioned` kinds keep versions (`texture_revision`, `publication_version`). `tombstone` marks a logical delete; `superseded_by` points forward.

5.3 The registry and the write gate (H-OG1, H-OG2)

`objectgraph.DefaultRegistry` lists every legal object kind with its store, identity mode, owner package (`OwnerVMStore`, `OwnerCorpusd`, `OwnerAutoradio`) and, where recorded, the Go type of its body; and every legal edge kind (`internal/objectgraph/registry.go`).

Until this week, the registry was consulted on **one** write path, `objectgraph.Service`. The guest store wrote almost everything through `ogStore.PutBatch/PutObject` directly (30 call sites), and corpusd's `ObjectGraphStore` and its HTTP routes never asked. A source scan found 26 object kinds and 12 edge kinds being written with no registration, including `choir.commitment_record` itself (doc: object graph registration gaps, H-OG1). Registration was a convention, not a check.

Commit 7f287a135 made it a check. `CheckWrite(objects, edges)` refuses any live object or edge whose kind the registry does not know (`ErrUnregisteredKind`), and both durable stores call it first in every mutation method. In the guest it runs before the vocabulary validator and before the projection interceptor, "so a refused write never reaches the event tape" (code comment, `registry.go`). Three deliberate exceptions:

- **tombstones pass**, so a historic object of a retired kind can still be retired;
- **replay** (the event projector) does not call it: refusing history would make the tape unreplayable;
- **vocabulary migrations** do not call it: they move existing rows.

That is a good gate design: strict on new writes, permissive on the past. `write_gate_test.go` and `registry_coverage_test.go` hold it, and a test named in the registry comment, `TestEveryRegisteredKindHasAWriter`, holds every non-reserved kind to a writer.

H-OG2 (427d14ffd) deleted the registration and constants of `choir.universal_wire_story_cluster`, a kind whose writer was deleted earlier and whose name is a retired product name. The frozen vocabulary decode table keeps the string, because old computers may still hold rows (doc: registration gaps problem, H-OG2). Deletion of a *write* path must never break *reading* old data.

5.4 Conditional writes

`PutBatchConditional(conditions, batch)` attaches preconditions to a batch: an `ObjectCondition` says an object must exist or not, and optionally must have a given `version_id` or `content_hash` (`internal/objectgraph/store.go`). On the direct-SQL path the store opens a serializable transaction, reads each named row `FOR UPDATE` in sorted ID order (a fixed lock order avoids deadlocks), compares, and returns `ErrConflict` on mismatch (`dolt_store.go`, `putBatch`). Management delegation uses this: the task lease is "a store precondition" checked in the command's one `PutBatchConditional` (commit be6e22238). See section 2.7 for the caveat on the intercepted path.

5.5 Where the graph is weak

- **Properties are untyped JSON in a `LONGTEXT`.** Any query on a property is a scan unless someone builds an index for that exact expression (sections 7 and 8).
- **Kinds are strings.** The gate now refuses unknown kinds, but a body's shape is checked only by Go decode at read time; `Schema` is advisory.
- **One shape for everything.** Captures, wakes, cursors, commitment records, Texture revisions and engineering claims share one table. Their access patterns are completely different: append-only audit, hot queue, point lookup.
- **Edges carry no confidence or provenance by default.** The claims design wants every link to carry who judged it and with what probability (doc: claims, events and entities §5.3).

6. Three stores: Store A, Store B and the guest

6.1 Get the names right

The names are easy to mix up, so fix them first. Both host stores run on Node B; the third lives inside each computer (code: `nix/node-b.nix`; doc: computer ontology, "Dolt Store Taxonomy"; state report §3.4).

store	where	Dolt mode	holds	who writes	in a computer's restore set?
Store A, platform control	host, `dolt sql-server` on 127.0.0.1:13306, database `platform`	server	event heads, append receipts (idempotency), lifecycle operations and receipts, checkpoints, file roots, replay watermarks, projection jobs and bases, route slots and certificates, key escrow, self-development modes	corpusd (mechanical CAS); vmctl for route slots (`VMCTL_ROUTE_DSN`, `internal/routeledger/sql_ledger.go`)	no: it is the authority restore is measured against
Store B, corpus	host, `dolt sql-server` on 127.0.0.1:13307, database `corpus`	server	the platform `og_objects` / `og_edges`: captures (`choir.web_capture`), publications and their versions, provenance, checks; source and ingestion tables. **The Autopaper object graph**	corpusd, authenticated as the writing computer	**no**: restoring it would rewind other computers
Guest embedded store	inside each VM, files on the data disk	embedded library	the computer's own `og_objects` / `og_edges` (runs, events, commitment records, Texture documents and revisions, wakes, cursors), `computer_event_index`, run memory	the guest's reducer, only by replaying or finalizing tape events	**yes**: restore is replay to an earlier head

The brief for this series called Store A "the guest" and Store B "the platform". That is not how the code or the ontology use the names. Store A and Store B are both host stores; the guest store has no letter.

6.2 Why Store A is small and boring on purpose

Store A is the one place where losing a write or accepting a wrong one corrupts a computer's identity. So it holds narrow tables with hard constraints (section 2.4) and does mechanical work only. The ontology is explicit: corpusd "mechanically performs authenticated event-head and platform-control CAS here; it does not select semantic events."

It did not start that way. Before the Store A/B split (19 September), the world-wire tables lived beside the event heads. On 9 October Store A still held about 17 million rows of abandoned wire data (`og_objects` 6.09M, `og_edges` 3.05M, `ingestion_events` 3.00M and more), 22 GB on disk, all in old chunk generations, and 86,021 Dolt commits (doc: Store A dead data and history). The record states the lesson that matters for any Dolt store: "Dropping them frees nothing. Dolt history retains every chunk ever committed ... Reclaiming needs a history rebuild (a fresh repo from current state)."

6.3 Why Store B is a separate process

The Nix configuration states the reason: Store B is a "separate process from the canonical event/control store (Store A, port 13306) so a corpus GC-OOM or bulk scan cannot take the canonical log down — the blast-radius isolation is the point of the split" (code: `nix/node-b.nix`, comment on `go-choir-corpus-dolt`). The corpus is big and grows fast: Store B held 8.76M objects and 4.38M edges when it was reset on 9 October, and captures now arrive at about 4,500 a day (doc: retrieval research §1.4). A heavy query or a garbage collection on that data must not stall event appends for every computer.

Inside corpusd, the split is a second connection pool. `platform.Store` holds `db` (Store A) and `corpusDB` (Store B), and `corpus()` returns the corpus pool; code that reads `og_*` tables "must use the corpus pool; reading them from Store A sees only pre-split leftovers" (code: `internal/platform/store.go`; doc: computer ontology). Each pool has its own debounced committer (`committer`, `corpusCommitter`).

6.4 Why the guest store is a cache

The guest's embedded store "indexes the externally pinned event chain and materializes the accepted effective state; it is never the sole durable event copy or an alternate head authority" (doc: computer ontology). The operational invariant behind it is O21, "a realization holds no unique state" (doc: guest store history bloat). If a guest's disk dies, the computer comes back by installing a verified projection base and replaying the tape tail. That is also why compaction may squash the guest store's Dolt history: nothing depends on it.

6.5 Why Store B is outside the restore set

A computer's restore "addresses an event head and restores the release pointer plus VM-local embedded state. Shared platform/world-wire Dolt (Store B: captures, publications) is OUT (restoring them would rewind other computers or shared services)" (doc: computer ontology). Publications are external effects, like sending an email: you correct them with a new forward action, not by rewinding. The claims design adds a careful consequence: every judgment written to Store B records the judging computer's event head, so if that computer is restored to an earlier head, its projection can choose to ignore its own later judgments while the judgments remain as history (doc: claims, events and entities §5.3).

7. Why reads got slow

7.1 The measurement

On 14 October at 16:46 UTC a 60-second Go CPU profile was taken of the paper computer's main `autoputer` process, with a host sample over the same window (doc: CPU efficiency §1b; evidence file `docs/evidence/computer-cpu-guest-profile-2026-10-14T1646Z.txt`). The numbers:

measure	value
main daemon CPU	1.17 cores (98% of guest time)
everything else in the guest, including 12 Yaegi workers	0.03 cores
model calls in the window	34; about 2.27 CPU-seconds per model call
under `database/sql.(*Rows).Next` (the embedded Dolt engine)	**73%**
under `encoding/json.Unmarshal`	40%
`listInboxRecordsBy` → `ogListAllByMetadata` → `ListObjectsByMetadataPage`	**52%**
↳ of which, the run-end round check (`closeRoundsOfEndedRun` → `OpenRounds` → `ConversationState`)	44%
claim pack per cell (`ListCommitmentClaimRecords` → `ListObjectsByOwnerAndBody`)	11%
model request build and reply parse	under 0.5%
Yaegi interpretation	about 2%

The paper's CPU was the store, not the agents. The CPU report's estimate of the floor for the work a model call actually needs is 30 to 60 ms today and 5 to 10 ms with a lean store, against 2.3 s spent (doc: CPU report, "The short answer").

7.2 Anatomy of the hot query

Every inbox, conversation and sender lookup ends in this query (`internal/objectgraph/dolt_store.go`, `ListObjectsByMetadataPage`):

```
SELECT canonical_id, object_kind, ..., body, metadata, ...
FROM og_objects
WHERE object_kind = ?
  AND JSON_UNQUOTE(JSON_EXTRACT(CAST(metadata AS JSON), ?)) = ?
  AND canonical_id > ?
ORDER BY canonical_id ASC LIMIT ?          -- page size 256 (ogMetadataPageSize)
```

Read it as the engine does:

1. **What can an index help with?** `object_kind = ?` and `canonical_id > ?`. Canonical IDs start with `obj:<kind>:`, so both narrow to a primary-key range: "all commitment records after the cursor".
2. **What can it not help with?** The JSON predicate. The JSON path is a bind parameter (`?`), so at planning time the engine does not even know which field is being compared. The column is `LONGTEXT`, so every row must be `CAST` to JSON (parsed), then the path extracted, then unquoted, then compared. No index on a column can serve an expression over that column unless the index was built on *that literal expression* (section 8.2).
3. **What does `LIMIT 256` do for a selective filter?** Very little. If a conversation has 12 records among 30,000 commitment records, the engine walks the range, parsing every row's JSON, until it has 256 matches or runs out of range. With 12 matches it runs out: the whole kind is scanned. The Go loop in `ogListAllByMetadataPageSize` (`internal/store/graph_store.go`) then asks for the next page after the last match, which scans the rest of the range again to find nothing. (**Inference** from the query shape and the canonical-ID layout; the profile confirms the cost, not the exact plan.)
4. **Then every matching row is decoded again in Go** (`decodeLifecycleObject`) and filtered by owner and computer (`listInboxRecordsBy`, `internal/store/inbox.go`).

7.3 The multiplier: what calls it, and how often

`OpenRounds` (`internal/store/inbox.go`) finds the conversations an inbox takes part in by listing records with `inbox = X` and then with `sender_inbox = X`: two scans. Then, for each conversation found, it calls `ConversationState`, which lists records with `conversation_id = C`: one more scan each. An inbox in *k* conversations costs *k + 2* scans of every commitment record ever written.

`closeRoundsOfEndedRun` (`internal/agentcore/inbox_records.go`) runs this at the end of every research and management run. Each cell also builds a claim pack by scanning the owner's commitment records for `$.directive` inside the *body* (`ListObjectsByOwnerAndBody`, which casts the `LONGBLOB` body to text and then to JSON per row), and reads its inbox by the same unindexed path.

So the cost model is:

```
cost per run end  ~  (k + 2) x R x (parse one row's JSON)
R = commitment records ever written   (captures alone add ~270/hour at 28 sources)
k = conversations the inbox has ever joined
```

Both *R* and *k* grow for the life of the computer, and neither depends on how much work the run did. The problem record's hypothesis, still unmeasured, is that "the paper's CPU per model call rises over the computer's life" (doc: guest CPU is unindexed store scans). This is the signature of a missing access path: cost proportional to history instead of to change.

And because every object graph call holds `engineMu` (section 4.5), a scan does not only burn CPU. It holds the one lock that the Texture editor, the health route and every desk need.

7.4 The half-wired replacement

The fix half exists. `metadataIndexColumns` maps three hot metadata fields (`run_id`, `trajectory_id`, `channel_id`) to stored generated columns, and `ensureMetadataColumns` creates them at boot with an index on `(object_kind, <column>, updated_at)` (`dolt_store.go`). Its comment records why: "measured under drain: JSON_EXTRACT full scans at ~1.4s/call". `ListObjectsByMetadata` uses the column when it exists. But:

- `ListObjectsByMetadataPage`, the paged variant every `ogListAllByMetadata` call uses, never looks at `metaColumns`;
- the fields the inbox needs (`inbox`, `sender_inbox`, `conversation_id`) have no column;
- `directive` is in the body, not the metadata.

This is a textbook case of the repository's convergence rule: "Before debugging subsystem X, search for a replacement ... If one exists and is not wired in, connecting it is preferred over patching" (CLAUDE.md, "Convergence Before Patching"). It also illustrates why a replacement gets stranded: the first fix was made at the call site that was measured, not at the access path all similar callers share.

7.5 One cause, three symptoms

The CPU report and problem record group three findings from one week under one substrate: "the object graph used as a JSON document store with scan-and-filter reads under one engine mutex" (doc: CPU report §1b):

1. the Texture document list going from 0.15 s to 1 to 2.7 s after idle (`internal/server/debug_pprof.go` header; doc: texture list cold latency);
2. 77% of the main heap under `AppendNewPayload` (doc: Autopaper intake defects);
3. this CPU profile.

Three symptoms in a week in one subsystem is the threshold at which the repository's rule says to stop patching and fix the substrate. Here the substrate fix is an indexed access path for each hot read.

8. Indexes and query plans, in general and in Dolt

8.1 Primer: what an index is

An **index** is a second copy of some columns of a table, kept sorted, with a pointer back to the full row (in InnoDB and Dolt, the primary key). It turns "find rows where X = 5" from a **scan** (read every row, test each) into a **seek** (walk a tree to the first 5, read forward while X = 5). A seek costs O(log n + matches); a scan costs O(n).

Rules that carry across almost every engine:

- **Composite indexes are sorted by their columns in order.** An index on `(kind, inbox, inbox_seq)` serves `kind = ?`, `kind = ? AND inbox = ?`, and `kind = ? AND inbox = ? AND inbox_seq > ?`, in that order. It does not serve `inbox = ?` alone. This is the **leftmost prefix** rule.
- **Equality columns first, then the range or sort column.** A range on an early column stops the index from narrowing later ones.
- **A covering index** contains every column the query needs, so the engine never visits the table. A non-covering seek does one extra lookup per matching row.
- **An index can avoid a sort** if its order matches `ORDER BY`. Then `LIMIT n` can stop after n rows. If the engine sorts instead (a TopN), it reads the whole range first.
- **Expressions defeat plain indexes.** `WHERE LOWER(title) = ?` cannot use an index on `title`. You need an index on the expression itself: a *functional index*, or a *generated column* holding the expression's value, indexed.
- **Selectivity decides whether an index is worth it.** If 90% of rows match, a scan is cheaper than 90% random lookups.
- **Indexes cost writes.** Every insert updates every index.

8.2 How Dolt does it

In Dolt, a secondary index is another prolly tree keyed by the indexed columns followed by the primary key (upstream; **inference** for exact internal layout). Generated columns come in two kinds: `STORED` (the value is computed at write time and kept in the row, so it can be indexed) and virtual. Choir's `ensureMetadataColumns` uses `GENERATED ALWAYS AS (JSON_UNQUOTE(JSON_EXTRACT(CAST(metadata AS JSON), '$.run_id'))) STORED`, with the path written literally into the DDL (code: `dolt_store.go`).

The retrieval research probed Dolt 2.1.9, the version Store B runs, and found **functional indexes work** on a JSON path over `LONGTEXT` (a hidden generated column; `EXPLAIN` shows `IndexedTableAccess`), "provided the query uses the same literal expression: today's code passes the path as a bind parameter and wraps `CAST(metadata AS JSON)`, which no index can match" (doc: retrieval research §3.1). The rule to remember: **the planner matches indexes to expressions by text, at plan time. A value supplied later cannot select an index.**

8.3 Reading a plan

`EXPLAIN` (Dolt also accepts `EXPLAIN PLAN`) prints the operator tree the engine will run. The node names worth knowing in go-mysql-server (upstream; names as seen in Choir's reports):

node	meaning	good or bad
`IndexedTableAccess(table on [index])`	a seek or range on that index	usually good; check it is the index you meant
`Table` / a bare table scan	read every row	bad on large tables
`Filter(expr)` above a scan	the predicate is tested per row	fine on few rows; the cost is the rows below it
`TopN(limit; sort)`	sort the whole input, keep N	bad if the input is the whole range
`LookupJoin`	for each left row, seek the right side by index	good for small left sides
`HashJoin`	build a hash table of one side	bad when that side is millions of rows
`MergeJoin`	walk two sorted inputs together	fine if both are small or bounded

8.4 Choir's planner incidents, as lessons

1. **The ignored unique index.** The replay page query `WHERE computer_id=? AND sequence>? ORDER BY sequence LIMIT ?` should have used the unique index `(computer_id, sequence)`. Dolt chose the primary key `(computer_id, idempotency_key)`, used only its `computer_id` prefix, read all 576,855 rows of the computer, filtered, and sorted. `FORCE INDEX` did not change the plan. Rewriting the range as point lookups, `sequence IN (a+1, ..., a+n)`, used the right index: 0.02 s instead of 3.6 s (doc: replay page scans; the shipped query is in `internal/platform/event_replay.go`). Lesson: when a planner will not do a range, an `IN` list of points on a dense key is a reliable workaround.
2. **`ORDER BY ... DESC LIMIT` is a full-range sort.** Over an equality prefix, Dolt 2.1.9 did a TopN over the whole range rather than walking the index backwards: 6 to 15 s on 700k rows. Adding an index alone made one window query *slower* (14.7 s). Fix: bound the time (`t >= now − window`) so the range is small (doc: retrieval research §4 G2).
3. **One index can wreck another query.** Adding `(kind, created_at)` made `to_id = ? AND kind = ?` pick it over `(to_id, kind)`: 1.5 ms became over 12 s. `USE INDEX`, `FORCE INDEX` and `ANALYZE TABLE` did not help; appending `to_id` to the new index restored the right plan (doc: retrieval research §4 G3).
4. **Join shape decides everything.** A plain `JOIN` became a `HashJoin` over 5M claims (11.8 s); `IN (subquery)` a `MergeJoin` over two full indexes (268 s); a CTE a `LookupJoin` (3.3 ms) (doc: retrieval research §4 G4).
5. **FULLTEXT is narrow.** It is used only when `MATCH` is the whole `WHERE`; it drops tokens under three characters ("AI", "EU", "US"), has no phrase or boolean mode, and is case-sensitive under the default binary collation (doc: retrieval research §3.1). Body text belongs in a derived SQLite FTS5 file, not in Dolt (§0 item 6).

The research's conclusion is the operational rule: every retrieval query that ships needs a **plan-shape test**, "a fixture repo and an assertion on `EXPLAIN PLAN` (index name, no `Table` full scan, no `HashJoin` on a large table), because one added index can silently turn a 1 ms query into a 12 s one" (doc: retrieval research §4 G). In a mature engine the planner is mostly trustworthy and plan tests are a luxury. In Dolt they are a correctness requirement.

8.5 What indexes cost in Dolt

Measured on a synthetic day at Store B scale (doc: retrieval research §4 F): 4,500 `og_objects` rows with a generated column and functional index took 4.4 s to write; 40,000 edges with five indexes took 22.2 s; a FULLTEXT index on 1M titles took 642 s to build and grew the repo to 20 GB before GC. Indexes are not free in any engine, and in a content-addressed engine every index write is also a set of new chunks.

9. Options and trade-offs

The levers below come from the CPU report §6 and the retrieval research §5, ordered as those reports order them. Each names its mutation class.

id	change	what it fixes	cost and risk	class
C1	`ListObjectsByMetadataPage` uses `metaColumns` like the unpaged read; add `inbox`, `sender_inbox`, `conversation_id` generated columns; verify with `EXPLAIN` that Dolt seeks	up to ~52% of the paper's main process	small; generated-column DDL at boot on existing stores (the code already skips a column whose expression fails on legacy rows)	orange
C2	fold only the conversations the ended run touched, or keep a `conversation_heads` projection	most of the 44% C1 leaves	small to medium; a projection on the tape path becomes red	orange (red if on the tape)
C3	promote `directive` to metadata with a column, or keep an open-claims projection	~11%	small	orange
C4	read a cell's inbox as an index range `(inbox, inbox_seq) > cursor`	~8%	small	orange
C6	per-run sequence high-water mark in `AppendEvent`	$O(n^2)$ per run	very small	red (Trace surface)
C7	compute canonical bytes once; drop the dry-run transaction; one platform call for pin + CAS	3 to 5% guest plus host	medium; changes the CAS protocol	red
typed projection table	one table for commitment records with typed, indexed `inbox`, `conversation_id`, `inbox_seq`, `directive`, maintained by the projector	subsumes C1-C4	larger; a second schema to migrate; must be rebuildable from the tape	red
plan-shape tests	assert `EXPLAIN` shape for each hot query	prevents silent regressions like the 12 s flip	needs fixture data	yellow
raise embedded connections, then remove `engineMu` in steps	lock contention	needs the race test first	orange	

Generated columns versus a typed table. Generated columns are the cheapest step: no new write path, the projector keeps writing one row, and Dolt maintains the column. Their limits are that every new hot field is a new column on a shared table, the JSON is still parsed on every write, and a table of 30 kinds carries columns that matter to one. A typed projection table is the long-term shape: it is how the ontology's "promote typed artifacts" rule looks in storage. It costs a schema of its own and a rebuild path from the tape. **Recommendation, from the reports:** do C1 to C4 now, each as a small slice profiled on the paper after deploy, and plan the typed table as the substrate fix that subsumes them.

On Dolt itself. None of today's evidence says "replace Dolt". It says: use Dolt for authority and history where its versioning earns its cost (Store B's audit, publication witnesses), keep rebuildable read models out of its history (`dolt_ignore`, or SQLite beside it), commit on a debounce, and test plans. The retrieval research's ladder ends with the same discipline for embeddings: measure on a frozen evaluation set, and add the next mechanism only when the current one stops moving the number.

Glossary

term	meaning
Access path	The way an engine reaches the rows a query needs: a scan, an index seek, a range.
Append-only log	A sequence of records where the only write is adding one at the end.
AS OF	Dolt SQL syntax to query a table as of a past commit, branch or tag.
Canonical JSON	One fixed byte encoding of a JSON value (RFC 8785), so hashes and signatures agree.
CAS (compare-and-swap)	"Set X to new only if it is still old"; the loser learns it lost.
Commitment record	Choir's typed precommit, report, resolve, disagreement or directive, stored as `choir.commitment_record`.
Content addressing	Naming data by the hash of its bytes, so identical data has one name and is stored once.
Covering index	An index holding every column a query needs, so the table is never visited.
Desired / effective head	What a computer has been told to become, versus what it has verifiably become.
DOLT_COMMIT	A Dolt procedure that records a commit in the version graph. Not needed for durability; SQL COMMIT is.
Engine mutex (`engineMu`)	The guest's process-wide lock around every embedded-Dolt object graph call.
Event sourcing	Storing the log of changes as truth and deriving state by replay.
Functional index	An index on an expression, such as a JSON path, rather than a plain column.
Generated column	A column whose value the database computes from an expression; `STORED` ones can be indexed.
Hash chain	Records that each include the previous record's hash, so the newest hash commits to all history.
Head	The hash (and sequence) of the newest record of a chain.
Idempotency key	A client-chosen operation ID stored with the result, so retries return the first result.
Keyset pagination	Paging by "rows after the last key seen" instead of `OFFSET`.
Materialized view / projection	A table derived from the log, kept up to date, rebuildable.
Object graph	Choir's typed objects and edges in `og_objects` and `og_edges`, with a kind registry.
Plan-shape test	A test that asserts the operator tree `EXPLAIN` prints for a query.
Precommitment	A prediction recorded before the evidence that resolves it exists.
Projection base	A verified, content-addressed snapshot of the guest store at a tape position, used to skip replay.
Prolly tree	Dolt's content-defined, hash-addressed B-tree variant, giving cheap diffs and shared history.
Store A	The host's platform control Dolt (:13306): heads, receipts, lifecycle, routes.
Store B	The host's corpus Dolt (:13307): the Autopaper object graph.
Tape	Choir's per-computer canonical event chain.
TopN	A plan node that sorts its whole input and keeps the first N rows.
Two-phase commit (local)	Write a "prepared" record, perform the external commit, then mark "finalized"; recover by checking which happened.
Write gate	`objectgraph.CheckWrite`: the one check every durable object-graph write passes.

Self-check

1. A guest appends an event; the host's CAS commits; the guest crashes before finalizing its projection. What does the next boot do, and which property of the host makes that safe?
2. Why can Dolt's garbage collector not reclaim the space used by old row versions in a store that ran `DOLT_COMMIT` after every write, even after the rows are deleted?
3. Store B is outside a computer's restore set but the guest store is inside it. Give the reason for each, in one sentence.
4. Explain, using the query text, why `JSON_UNQUOTE(JSON_EXTRACT(CAST(metadata AS JSON), ?)) = ?` cannot use an index, and why `LIMIT 256` does not bound its cost.
5. You add a composite index for a new query and an unrelated query becomes 1,000 times slower. What happened, and what practice would have caught it before deploy?

Answer sketches

1. `RecoverPrepared` finds the `prepared` row, reads the host head, sees it equals this event, and calls `CompareAndSwap` again. The host's `(computer_id, idempotency_key)` receipt lookup returns the original signed receipt instead of appending twice; the guest verifies and finalizes. Idempotency makes a retried commit safe.
2. GC keeps every chunk reachable from any commit. Each commit's root hash reaches the chunks of that version, so as long as the commits exist, every past version is live. Only rebuilding history (a fresh repository, or squashing) frees them; hence the debounced committer.
3. Store B is shared: rewinding it would rewind other computers and published facts. The guest store is a cache of the computer's own tape, so restoring it is just replay to an earlier head.
4. The path is a bind parameter and the column is wrapped in a `CAST`, so no index built on any literal expression matches; every row's JSON is parsed. A selective filter finds fewer than 256 matches, so the engine reads to the end of the range anyway, and the pager then reads the rest of the range again to confirm there is nothing more.
5. The planner chose the new index for the other query because its leading columns matched; the chosen index could not serve that query's predicate well (Choir's case: `(kind, created_at)` chosen over `(to_id, kind)`). A plan-shape test asserting the expected index on each hot query would have failed in CI.

Read these three files

1. **`internal/computerevent/appender.go`**: read `appendLocked` and `RecoverPrepared`. Look for where the guest refuses to write on a stale projection, where the two-phase prepare and finalize happen, and how recovery uses the host's idempotency.
2. **`internal/platform/computer_events.go`**: read the schema DDL and `ComputerEventCAS.CompareAndSwap`. Count the independent defences (recomputed commitments, striped lock, idempotency, serializable transaction, conditional update, unique keys) and ask which one would catch which bug.
3. **`internal/objectgraph/dolt_store.go`**: read `SchemaDDL`, `metadataIndexColumns`, `ListObjectsByMetadata` next to `ListObjectsByMetadataPage`, and `putBatch`. The difference between the two list functions is today's CPU finding in twenty lines.

Sources and confidence

- **Code read at 980ffaf4e:** `internal/computerevent` (`event.go`, `reducer.go`, `appender.go`, `canonical.go`), `internal/platform` (`computer_events.go`, `store.go`, `event_replay.go`), `internal/objectgraph` (`dolt_store.go`, `registry.go`, `object.go`, `store.go`, `engine_mutex.go`), `internal/store` (`projection_tape.go`, `store.go`, `inbox.go`, `graph_store.go`, `texture.go`, `computer_events.go`), `internal/agentcore/inbox_records.go`, `internal/types/commitment.go`, `internal/doltbatch/`

`committer.go`, `internal/projectionbase` (doc comments), `internal/routeledger/sql_ledger.go`, `nix/node-b.nix`, `go.mod`. The claim that no production code uses `AS OF`, history tables, branches, merges or tags is from a text search of non-test files under `internal/` and `cmd/`.

- **Measured numbers** are from the cited reports and problem records, not re-measured here: computer CPU efficiency, guest CPU is unindexed store scans, retrieval research, replay page scans, 218 GB history, Store A dead data, guest store history bloat, embedded Dolt single connection, object graph registration gaps.
- **Doctrine and design:** Choir doctrine, computer ontology, state and vision, claims, events and entities, Precommitment Records — Engineering Memo.
- **Upstream:** Dolt's documentation on prolly trees and querying history; general B+tree and InnoDB behaviour is standard database material. The exact internal layout of Dolt secondary indexes is stated from general knowledge and marked inference.
- **Inferences to test:** the check-then-act window between `evaluateConditions` and the projection interceptor (section 2.7); the exact plan of the paged metadata query and the second-scan behaviour of the pager (section 7.2); the write-amplification estimate (section 4.2); concurrency safety of raising embedded connections (section 4.6).
- **Related documents in this series:** the desk runtime (inboxes, wakes and runs that produce these records), the computer lifecycle (restore, refresh and the planned-restart marker), publication and Autopaper (what Store B holds), and reasoning about trade-offs (single state authority and idempotency as general tools).